

PromptTrap MVP

Windows-First Learning Handoff

Build a small, local document-safety scanner with repeatable tests and clear evidence.

Topic	Instruction
Purpose	Learning project: document parsing, OCR, Unicode inspection, rule-based detection, sanitization, and metrics.
Primary machine	Develop on Windows. Recommended: VS Code + WSL2 Ubuntu + Docker Desktop WSL integration.
Portability goal	A fresh macOS clone should run with the same Docker Compose commands.
Cost target	\$0 development cost. No paid APIs, no private datasets, no cloud requirement.

1. MVP scope

Build PromptTrap as a local-first scanner. It should accept PDF, DOCX, HTML, and TXT files; detect hidden or machine-readable manipulation; show evidence; and create a clean AI-readable output.

Must have	Output
CLI scan command	Scans one file and writes an output folder.
Human vs machine view	Shows visible/OCR text compared with parser-extracted text.
Known issue detection	Hidden text, tiny/off-page text, zero-width chars, metadata/comments, hidden HTML/CSS, prompt-like rules.
Safe output	safe_text.txt and safe_payload.json, with SHA-256 hashes.
Benchmark	Synthetic clean/attacked files plus metrics report.

Out of scope: accounts, external integrations, polished UI, truth verification, AI-writing detection, paid LLM calls, and production deployment.

2. Windows setup and macOS portability

- Install: Git for Windows, VS Code, Docker Desktop, WSL2 Ubuntu, Python 3.12+, and uv.
- Keep the repo inside the WSL Linux filesystem, for example ~/code/prompttrap, not OneDrive or C:\Users\... .
- Use Docker Compose as the official run path. Put Tesseract, LibreOffice, qpdf, ExifTool, and fonts inside Docker.
- Use pathlib.Path everywhere. Do not hardcode Windows or macOS paths.
- Keep helper commands in Python scripts so they run on Windows, macOS, and Linux.

Windows/WSL quick start

```
mkdir -p ~/code && cd ~/code
git clone <repo-url> prompttrap
cd prompttrap
docker compose up --build
docker compose run --rm app pytest
docker compose run --rm app python -m prompttrap scan samples/attacked_resume.pdf --out out/demo
```

macOS should use the same pattern: clone the repo, run docker compose up --build, run pytest, then run the benchmark command.

Repo rule	Reason
Commit Dockerfile, compose.yaml, pyproject.toml, uv.lock, README.md, .env.example, .gitattributes, .gitignore.	Fresh clones should be reproducible.
Do not commit .venv, generated benchmark data, out/, .DS_Store, __pycache__, or local secrets.	Keeps repo clean and portable.
Use UTF-8 and LF line endings for source files.	Avoids Windows/macOS diff noise.

3. Simple architecture

```
prompttrap/  
  prompttrap/scanner/      # PDF, DOCX, HTML, TXT scanners  
  prompttrap/sanitizer/    # safe_text and safe_payload  
  prompttrap/benchmark/    # generator, attack seeder, metrics  
  prompttrap/reports/      # JSON/HTML evidence reports  
  samples/                 # tiny clean + attacked examples  
  tests/                   # pytest unit and integration tests  
  apps/viewer/             # optional tiny Streamlit viewer
```

Need	Suggested tools
PDF	pdfminer.six, pypdf, pypdfium2, pikepdf
DOCX	python-docx plus direct zipfile/lxml XML inspection
HTML/TXT	beautifulsoup4, lxml, tinycss2, regex
OCR/rendering	Tesseract + LibreOffice inside Docker; pillow + pytesseract in Python
Diff/score	rapidfuzz, regex, hashlib, json, pathlib
Benchmark/tests	faker, reportlab, pytest, ruff
Optional viewer	Streamlit only after CLI and benchmark work

Avoid adding large frameworks early. The CLI scanner and benchmark matter more than a dashboard.

4. Scanner methodology

- Hash original file with SHA-256 and detect type by content, not only extension.
- Extract machine-readable content: text, coordinates when available, metadata, comments, hidden XML parts, alt text, annotations, and suspicious encoded strings.
- Render/OCR where practical to approximate what a human would see.
- Compare machine-extracted text with visible/OCR text and flag mismatches.
- Apply deterministic rules for hidden text, Unicode tricks, metadata/comments, hidden CSS, base64-like strings, and prompt-like language.
- Generate safe_text.txt and safe_payload.json from visible or sanitized text, then re-scan to confirm seeded payloads are gone.

Issue code family	Examples to catch
PDF	white/tiny/off-page text, metadata prompts, annotations, OCR mismatch
DOCX	comments, headers/footers, metadata, hidden runs, alt text
HTML	display:none, visibility:hidden, opacity:0, comments, meta tags, alt/title text
General	zero-width Unicode, base64-like payloads, prompt-like instructions

5. Benchmark and metrics

Build the benchmark before polishing the viewer. It is the proof that the scanner is improving.

Benchmark set	Squeezed MVP count
Clean synthetic resumes	100
Attacked synthetic resumes	100
Clean synthetic generic docs	50
Attacked synthetic generic docs	50

Metric	Target by Day 10
Seeded attack recall	$\geq 90\%$
False positives on clean synthetic docs	$\leq 3\%$
Sanitization leakage	0 seeded payloads remain in safe outputs
Visible content preservation	$\geq 95\%$
Crash rate	$\leq 1\%$ on generated benchmark set

Required commands

```
python -m prompttrap benchmark generate --clean 150 --attacked 150
python -m prompttrap benchmark run --input data/generated --out reports/benchmark.json
python -m prompttrap scan samples/attacked_resume.pdf --out out/demo
```

6. Shortest practical timeline

Target 10 focused working days. Use Days 11-15 only as a buffer for OCR, PDF/DOCX edge cases, or cleanup.

Days	Focus	Done when...
1	Repo, Docker, CLI skeleton	Fresh clone builds; pytest and ruff run; scan command exists.
2	Synthetic generator	Clean and attacked PDF/DOCX/HTML/TXT samples plus manifest.
3-4	PDF scanner + OCR mismatch	PDF hidden/tiny/off-page/metadata/zero-width issues detected and reported.
5	Safe outputs + report schema	safe_text.txt, safe_payload.json, report.json, and hashes are written.
6	DOCX scanner	Comments, metadata, headers/footers, hidden runs, and alt text are scanned.
7	HTML/TXT scanner	Hidden CSS/comments/meta plus TXT Unicode/prompt-like checks work.
8	Benchmark metrics	Recall, false positives, leakage, preservation, crash rate, and timing are printed.
9	Tiny viewer or evidence.html	Upload/view one file locally, or generate a clear evidence.html page.
10	Portability check + handoff	README has Windows/macOS quick starts, known limits, and sample outputs.

7. AI use is encouraged

Use AI actively to learn faster. Ask for explanations, small functions, tests, refactors, and debugging help. Review everything before committing.

Good use	Boundary
Ask AI to explain tracebacks, PDF/DOCX structures, and library behavior.	Do not paste private or real submitted documents into AI tools.
Ask AI to write pytest cases for each detector.	Do not let AI invent benchmark results. Metrics must come from local runs.
Ask AI to check pathlib and Windows/macOS portability.	Do not add paid APIs or cloud dependencies without approval.
Ask AI to review small functions and suggest simplifications.	Do not copy large blocks blindly without understanding and tests.

Useful prompts

```
"Write pytest tests for this detector, including Windows path edge cases."  
"Review this function for pathlib and cross-platform bugs."  
"Explain this DOCX XML part and where hidden text might live."  
"Given this traceback, suggest the smallest fix and a regression test."
```

8. Definition of done

- Windows/WSL clone runs with Docker Compose and passes tests.
- macOS clone uses the same Docker Compose commands.
- CLI scans PDF, DOCX, HTML, and TXT sample files.
- At least one attacked sample clearly shows human-visible vs machine-extracted mismatch.
- Every scan writes report.json, safe_text.txt, safe_payload.json, original hash, safe hash, issue list, and processing time.
- Benchmark outputs recall, false-positive rate, sanitization leakage, preservation, crash rate, and timing.
- README explains setup, commands, tests, metrics, known limitations, and next steps in plain language.

Final advice: make one detector work end-to-end before adding more. Prefer simple, readable code with tests over clever code without proof.